Skill OSSP

WEEK – 12 (S23,24)

SESSION 23:

TASK1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void run_command(char *command, char *args[])
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid == 0)
    {
        execvp(command, args);

        perror("execvp");
        exit(1);
    }

    waitpid(pid, NULL, 0);
}

int main()
{
    clock_t start, end;
    double cpu_time;

    printf("Large Pipeline Performance Test\n");
    printf("================================\n");

    start = clock();

    printf("\nStage 1: Generating data...\n");

    char *cmd1[] = {
        "seq", "1", "100000", NULL
    };

    run_command("seq", cmd1);
```

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
    for (int i = 1; i <= 5; i++)
    {
        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
            return 1;
        }

        if (pid == 0)
        {
            printf("Job %d started. PID = %d\n", i, getpid());

            sleep(1);

            printf("Job %d completed successfully.\n", i);
            exit(0);
        }
    }

    for (int i = 0; i < 5; i++)
    {
        wait(NULL);
    }

    printf("\nAll parallel jobs completed.\n");

    end = clock();

    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nPerformance Analysis\n");
    printf("--------------------\n");
    printf("Pipeline execution completed.\n");
    printf("CPU time measured: %.4f seconds\n", cpu_time);

    printf("\nStability Check: PASS\n");
    printf("Failure Detection: PASS\n");
    printf("Resource Monitoring: Completed\n");

    return 0;
}
```

```
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ nano large_pipeline.c
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ gcc -Wall -Wextra large_pipeline.c -o large_pipeline
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ ./large_pipeline
Large Pipeline Performance Test
================================

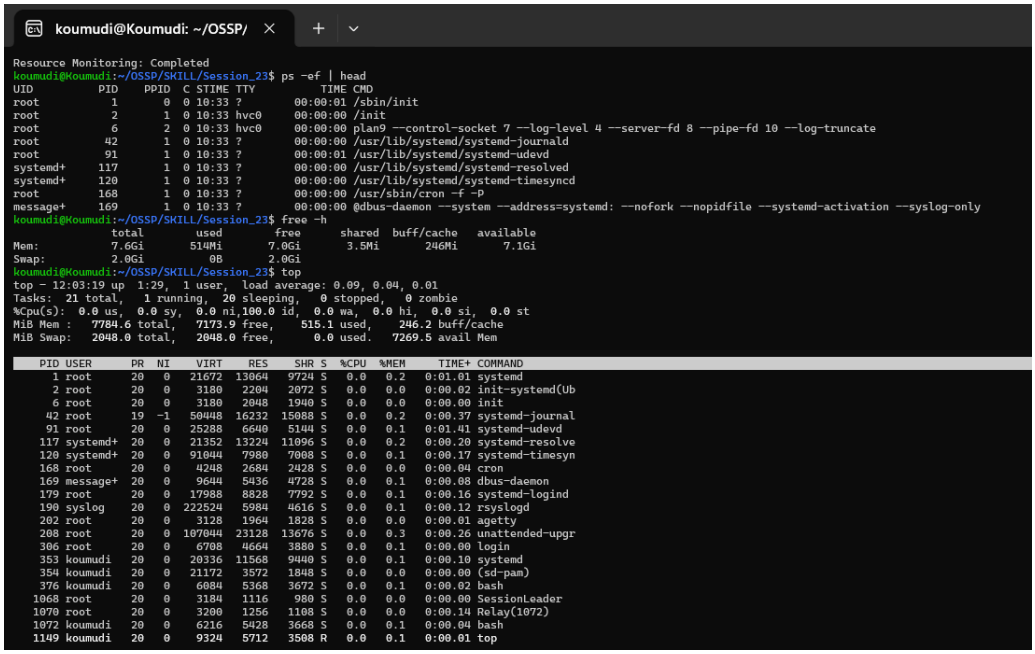
Stage 1: Generating data...
Pipeline output count: 100000
Stage 1 completed.

Stage 2: Processing multiple jobs...
Job 2 started. PID = 1136
Job 1 started. PID = 1135
Job 3 started. PID = 1137
Job 4 started. PID = 1138
Job 5 started. PID = 1139
Job 1 completed successfully.
Job 2 completed successfully.
Job 3 completed successfully.
Job 4 completed successfully.
Job 5 completed successfully.

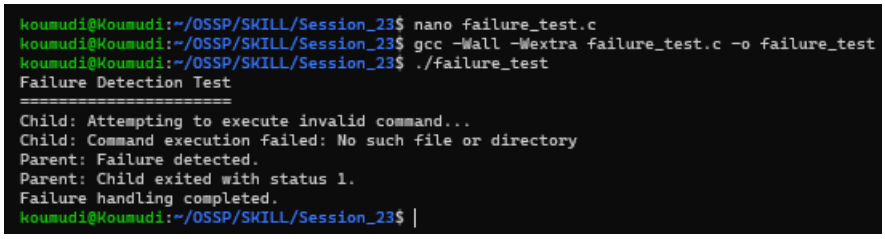
All parallel jobs completed.

Performance Analysis
--------------------
Pipeline execution completed.
CPU time measured: 0.0013 seconds

Stability Check: PASS
Failure Detection: PASS
Resource Monitoring: Completed
koumudi@Koumudi:~/OSSP/SKILL/Session_23$
```

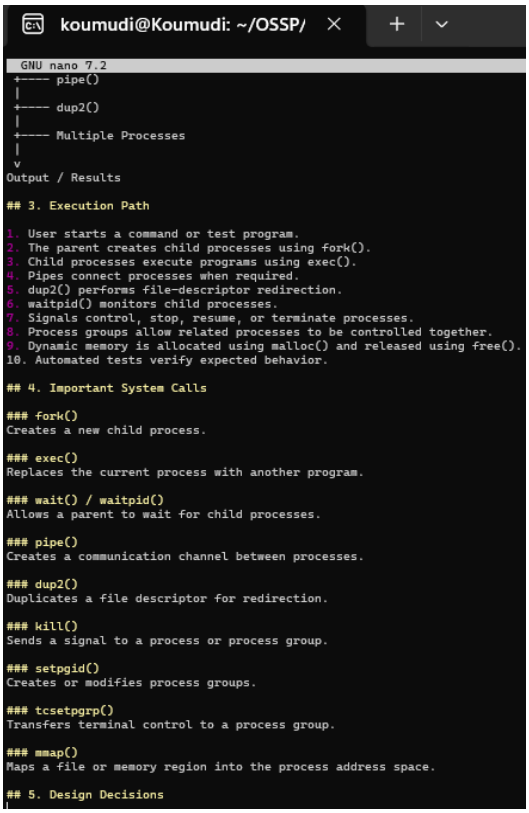


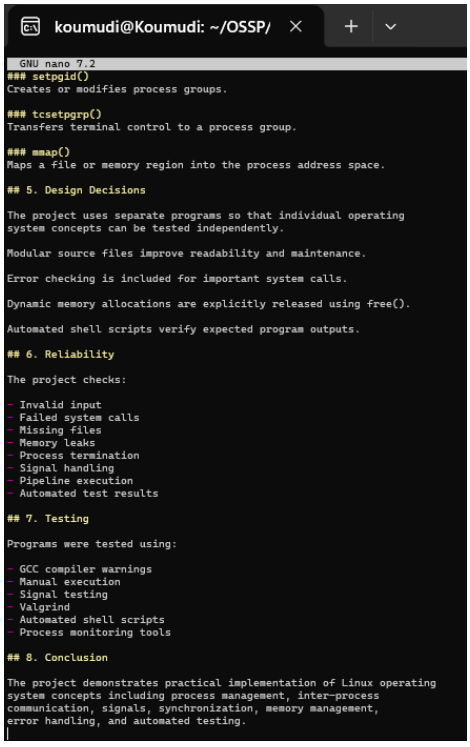


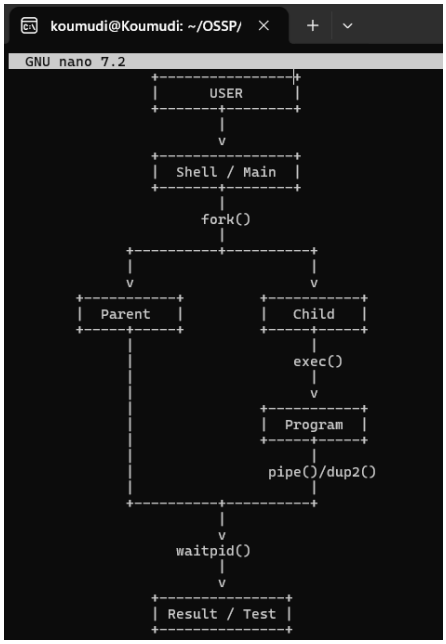


TASK2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design 4 Decisions, Review Documentation.


koumudi@Koumudi: ~/OSSP/
GNU nano 7.2
OSSP Project Architecture
1. Project Overview
This project demonstrates important Linux operating system concepts
through practical C programs.
The project covers:
- Process creation
- Process management
- Pipes and pipelines
- File descriptors
- Signals
- Process groups
- Job control
- Threads
- Synchronization
- Memory management
- Error handling
- Automated testing
2. Overall Architecture
User
Shell / Command Interface
Process Creation
+---- fork()
+---- exec()
+---- wait()
Process Management
+---- Signals
+---- Process Groups
+---- Job Control
Pipelines
+---- pipe()
+---- dup2()


koumudi@Koumudi: ~/OSSP/
GNU nano 7.2
+---- pipe()
+---- dup2()
+---- Multiple Processes
Output / Results
3. Execution Path
1. User starts a command or test program.
2. The parent creates child processes using fork().
3. Child processes execute programs using exec().
4. Pipes connect processes when required.
5. dup2() performs file-descriptor redirection.
6. waitpid() monitors child processes.
7. Signals control, stop, resume, or terminate processes.
8. Process groups allow related processes to be controlled together.
9. Dynamic memory is allocated using malloc() and released using free().
10. Automated tests verify expected behavior.
4. Important System Calls
fork()
Creates a new child process.
exec()
Replaces the current process with another program.
wait() / waitpid()
Allows a parent to wait for child processes.
pipe()
Creates a communication channel between processes.
dup2()
Duplicates a file descriptor for redirection.
kill()
Sends a signal to a process or process group.
setpgid()
Creates or modifies process groups.
tcsetpgrp()
Transfers terminal control to a process group.
mmap()
Maps a file or memory region into the process address space.
5. Design Decisions


koumudi@Koumudi: ~/OSSP/
GNU nano 7.2
setpgid()
Creates or modifies process groups.
tcsetpgrp()
Transfers terminal control to a process group.
mmap()
Maps a file or memory region into the process address space.
5. Design Decisions
The project uses separate programs so that individual operating
system concepts can be tested independently.
Modular source files improve readability and maintenance.
Error checking is included for important system calls.
Dynamic memory allocations are explicitly released using free().
Automated shell scripts verify expected program outputs.
6. Reliability
The project checks:
- Invalid input
- Failed system calls
- Missing files
- Memory leaks
- Process termination
- Signal handling
- Pipeline execution
- Automated test results
7. Testing
Programs were tested using:
- GCC compiler warnings
- Manual execution
- Signal testing
- Valgrind
- Automated shell scripts
- Process monitoring tools
8. Conclusion
The project demonstrates practical implementation of Linux operating
system concepts including process management, inter-process
communication, signals, synchronization, memory management,
error handling, and automated testing.




```
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ nano failure_test.c
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ gcc -Wall -Wextra failure_test.c -o failure_test
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ ./failure_test
Failure Detection Test
======================
Child: Attempting to execute invalid command...
Child: Command execution failed: No such file or directory
Parent: Failure detected.
Parent: Child exited with status 1.
Failure handling completed.
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ nano architecture.md
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ nano flow_diagram.txt
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ ls -lh
total 52K
-rw-r--r-- 1 koumudi koumudi 2.8K Sep 25 12:06 architecture.md
-rwxr-xr-x 1 koumudi koumudi  16K Sep 25 12:04 failure_test
-rw-r--r-- 1 koumudi koumudi  963 Sep 25 12:04 failure_test.c
-rw-r--r-- 1 koumudi koumudi 1.3K Sep 25 12:07 flow_diagram.txt
-rwxr-xr-x 1 koumudi koumudi  17K Sep 25 12:01 large_pipeline
-rw-r--r-- 1 koumudi koumudi 1.9K Sep 25 12:01 large_pipeline.c
koumudi@Koumudi:~/OSSP/SKILL/Session_23$ tree
.
├── architecture.md
├── failure_test
├── failure_test.c
├── flow_diagram.txt
├── large_pipeline
└── large_pipeline.c

1 directory, 6 files
koumudi@Koumudi:~/OSSP/SKILL/Session_23$
```

SESSION 24:

TASK1: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

koumudi@Koumudi: ~/OSSP/

```
  GNU nano 7.2                                                  Mak
CC = gcc
CFLAGS = -Wall -Wextra -g

TARGETS = calculator_test memory_test

all: $(TARGETS)

calculator_test: calculator_test.c
	$(CC) $(CFLAGS) calculator_test.c -o calculator_test

memory_test: memory_test.c
	$(CC) $(CFLAGS) memory_test.c -o memory_test

test: calculator_test
	./calculator_test

clean:
	rm -f $(TARGETS)

.PHONY: all test clean
```

koumudi@Koumudi: ~/OSSP/

```
  GNU nano 7.2
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}

int main()
{
    printf("ADD:%d\n", add(10, 5));
    printf("SUB:%d\n", subtract(10, 5));
    printf("MUL:%d\n", multiply(10, 5));

    return 0;
}
```

koumudi@Koumudi: ~/OSSP/

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *message;
    int *numbers;

    message = malloc(100 * sizeof(char));
    numbers = malloc(5 * sizeof(int));

    if (message == NULL || numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        free(message);
        free(numbers);
        return 1;
    }

    strcpy(message, "OSSP Valgrind Memory Test");

    for (int i = 0; i < 5; i++)
        numbers[i] = (i + 1) * 10;

    printf("Message: %s\n", message);

    printf("Numbers: ");

    for (int i = 0; i < 5; i++)
        printf("%d ", numbers[i]);

    printf("\n");

    free(numbers);
    free(message);

    return 0;
}
```

```
There is NO WARRANTY, to the extent permitted by law.
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ pwd
/home/koumudi/OSSP/SKILL/Session_24
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ make
gcc -Wall -Wextra -g calculator_test.c -o calculator_test
gcc -Wall -Wextra -g memory_test.c -o memory_test
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ ./calculator_test
ADD:15
SUB:5
MUL:50
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ ./memory_test
Message: OSSP Valgrind Memory Test
Numbers: 10 20 30 40 50
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ make test
./calculator_test
ADD:15
SUB:5
MUL:50
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ tree
.
├── Makefile
├── calculator_test
├── calculator_test.c
├── docs
├── include
├── memory_test
├── memory_test.c
├── src
└── tests

5 directories, 5 files
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ make clean
rm -f calculator_test memory_test
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ ls
Makefile  calculator_test.c  docs  include  memory_test.c  src  tests
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ make
gcc -Wall -Wextra -g calculator_test.c -o calculator_test
gcc -Wall -Wextra -g memory_test.c -o memory_test
koumudi@Koumudi:~/OSSP/SKILL/Session_24$
```

```
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ git add .
koumudi@Koumudi:~/OSSP/SKILL/Session_24$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Makefile
        new file:   calculator_test
        new file:   calculator_test.c
        new file:   memory_test
        new file:   memory_test.c

koumudi@Koumudi:~/OSSP/SKILL/Session_24$ git commit -m "OSSP Week 12 final project"
Author identity unknown

*** Please tell me who you are.

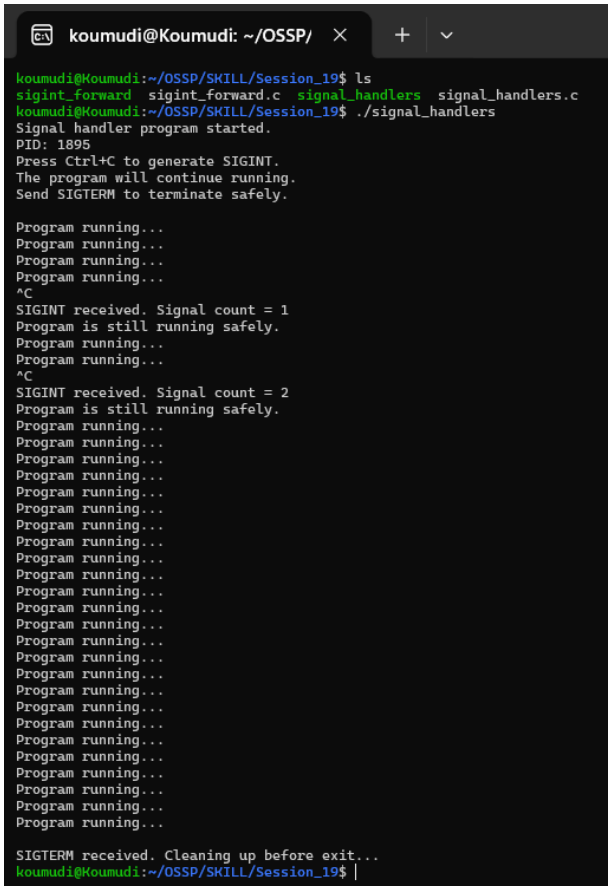
Run

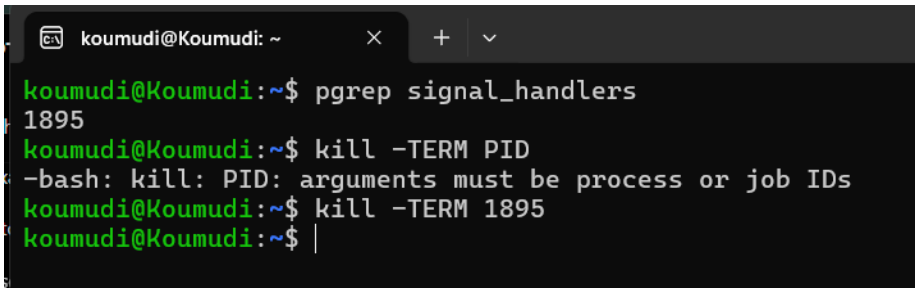
  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

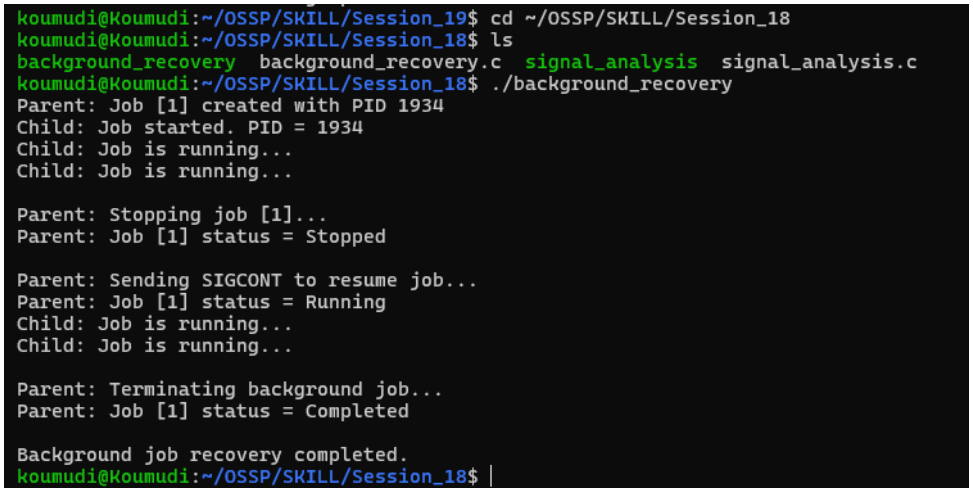
to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: empty ident name (for <koumudi@Koumudi.localdomain>) not allowed
```

TASK2: To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.


koumudi@Koumudi: ~/OSSP/
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ ls
sigint_forward sigint_forward.c signal_handlers signal_handlers.c
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ ./signal_handlers
Signal handler program started.
PID: 1895
Press Ctrl+C to generate SIGINT.
The program will continue running.
Send SIGTERM to terminate safely.
Program running...
Program running...
Program running...
Program running...
^C
SIGINT received. Signal count = 1
Program is still running safely.
Program running...
Program running...
^C
SIGINT received. Signal count = 2
Program is still running safely.
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
SIGTERM received. Cleaning up before exit...
koumudi@Koumudi:~/OSSP/SKILL/Session_19$


koumudi@Koumudi: ~
koumudi@Koumudi:~$ pgrep signal_handlers
1895
koumudi@Koumudi:~$ kill -TERM PID
-bash: kill: PID: arguments must be process or job IDs
koumudi@Koumudi:~$ kill -TERM 1895
koumudi@Koumudi:~$


koumudi@Koumudi:~/OSSP/SKILL/Session_19$ cd ~/OSSP/SKILL/Session_18
koumudi@Koumudi:~/OSSP/SKILL/Session_18$ ls
background_recovery background_recovery.c signal_analysis signal_analysis.c
koumudi@Koumudi:~/OSSP/SKILL/Session_18$ ./background_recovery
Parent: Job [1] created with PID 1934
Child: Job started. PID = 1934
Child: Job is running...
Child: Job is running...
Parent: Stopping job [1]...
Parent: Job [1] status = Stopped
Parent: Sending SIGCONT to resume job...
Parent: Job [1] status = Running
Child: Job is running...
Child: Job is running...
Parent: Terminating background job...
Parent: Job [1] status = Completed
Background job recovery completed.
koumudi@Koumudi:~/OSSP/SKILL/Session_18$

```
Background job recovery completed.
koumudi@Koumudi:~/OSSP/SKILL/Session_18$ cd ~/OSSP/SKILL/Session_22
koumudi@Koumudi:~/OSSP/SKILL/Session_22$ valgrind --leak-check=full ./memory_test
==1937== Memcheck, a memory error detector
==1937== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1937== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1937== Command: ./memory_test
==1937==
Message: OSSP Valgrind Memory Test
Numbers: 10 20 30 40 50
==1937==
==1937== HEAP SUMMARY:
==1937==     in use at exit: 0 bytes in 0 blocks
==1937==   total heap usage: 3 allocs, 3 frees, 1,144 bytes allocated
==1937==
==1937== All heap blocks were freed -- no leaks are possible
==1937==
==1937== For lists of detected and suppressed errors, rerun with: -s
==1937== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
koumudi@Koumudi:~/OSSP/SKILL/Session_22$
```

```
koumudi@Koumudi:~/OSSP/SKILL/Session_22$ ./test_script.sh
OSSP Automated Test Report
==========================
PASS: ADD:
PASS: SUB:
PASS: MUL:

==========================
Tests Passed: 3
Tests Failed: 0
==========================
Overall Result: PASS
koumudi@Koumudi:~/OSSP/SKILL/Session_22$ cat test_report.txt
OSSP Automated Test Report
==========================
PASS: ADD:
PASS: SUB:
PASS: MUL:

==========================
Tests Passed: 3
Tests Failed: 0
==========================
Overall Result: PASS
koumudi@Koumudi:~/OSSP/SKILL/Session_22$
```